WUOLAH

esquemas-de-controladores-y-midd...

RoelioIngeniero

Introducción a la Ingeniería del Software y los Sistemas de Información II

2º Grado en Ingeniería Informática - Ingeniería del Software

Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla

Accede al documento original

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

ESTRUCTURAS A MECANIZAR PARA EXAMEN BACKEND

MIDDLEWARES:

MODELO con Cálculo (Virtual Attribute) Average

```
// Dentro de la clase en models/Padre.js
async getAvgValue () { // [cite: 962]
  const hijos = await this.getHijos() // [cite: 964]
  if (hijos.length === 0) return 0 // [cite: 654, 965]

  const suma = hijos.reduce((acc, hijo) => acc + hijo.valor, 0) //
[cite: 967]
  return suma / hijos.length // [cite: 968]
}
```

checkBelongsTo

```
// Ejemplo: checkReviewBelongsToRestaurant
export const checkReviewBelongsToRestaurant = async (req, res, next) =>
{
  try {
    const review = await Review.findByPk(req.params.reviewId) [cite:
935]

    // Comparamos si el ID del padre (restaurante) coincide con el de
la URL
    if (review.restaurantId.toString() ===
req.params.restaurantId.toString()) { [cite: 804]
      return next()
    }

    // Si no coincide, error 409 Conflict o 403 según pida el PDF
[cite: 620, 643, 1005]
    return res.status(409).send('This review does not belong to this
restaurant.')
  } catch (err) {
    return res.status(500).send(err.message)
  }
```

Lógica de negocio cruzada (Historial / Duplicados)

Comprueba si el cliente puede o no realizar la acción basándose en otra tabla (Pedidos).

```
// A. ¿Ha comprado antes? [cite: 677, 923]
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
export const userHasPlacedOrderInRestaurant = async (req, res, next) =>
{
  const ordersCount = await Order.count({
    where: { userId: req.user.id, restaurantId: req.params.restaurantId
} [cite: 926]
  })
  if (ordersCount === 0) {
    return res.status(409).send('User cannot review without orders.')
[cite: 929]
  }
  next()
}

// B. ¿Ya ha dejado una review? [cite: 678, 931]
export const checkCustomerHasNotReviewed = async (req, res, next) => {
  const existingReview = await Review.findOne({
    where: { restaurantId: req.params.restaurantId, customerId:
req.user.id } [cite: 936]
  })
  if (existingReview) {
    return res.status(409).send('You have already reviewed this
restaurant.') [cite: 954]
  }
  next()
}
```

Check Ownership

```
// Ejemplo: checkShippingAddressOwnership o checkReviewOwnership
export const checkOwnership = async (req, res, next) => {
  try {
    // 1. Buscamos la entidad por el ID que viene en la URL
    const entity = await Modelo.findByPk(req.params.entityId) [cite:
935]

    // 2. Comparamos el dueño en la BD con el ID del usuario logueado
    if (req.user.id === entity.userId) { // A veces es .customerId
[cite: 591, 598]
      return next() // Todo ok, pasa al siguiente paso
    }

    // 3. Si no es el dueño, error 403 Forbidden [cite: 122, 140, 151,
408, 416, 609, 618]
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

```
    return res.status(403).send('Not enough privileges. This is not
yours.')
  } catch (err) {
    return res.status(500).send(err.message)
  }
}
```

CONTROLADORES

1. Interruptor unidireccional (markDefault / activate)

```
async markDefault (req, res) {
  try {
    // 1. "Apagar" todas las del usuario (Reset masivo)
    await Modelo.update(
      { isDefault: false },
      { where: { userId: req.user.id } }
    )
    // 2. "Encender" solo la que indica la URL (ID)
    await Modelo.update(
      { isDefault: true },
      { where: { id: req.params.modeloId } }
    )
    // 3. Recuperar y devolver la entidad actualizada
    const updated = await Modelo.findByPk(req.params.modeloId)
    res.json(updated)
  } catch (err) {
    res.status(500).send(err.message)
  }
}
```

2. Creación lógica de “primer elemento” (create)

```
async create (req, res) {
  try {
    // 1. Contar cuántos registros tiene ya el usuario
    const count = await Modelo.count({
      where: { userId: req.user.id }
    })
    // 2. Construir el nuevo objeto con los datos del body y el userId
    const newEntity = Modelo.build(req.body)
    newEntity.userId = req.user.id
    // 3. Si el contador es 0, aplicar la lógica especial
    if (count === 0) {
      newEntity.isDefault = true
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio

Necesito concentración

ali ali oooh esto con 1 coin me lo quito yo...

WUOLAH

```
    }
    // 4. Guardar y devolver
    const saved = await newEntity.save()
    res.json(saved)
  } catch (err) {
    res.status(500).send(err.message)
  }
}
```

3. Listado de ordenación y filtrado (index)

```
async index (req, res) {
  try {
    const entities = await Modelo.findAll({
      // Filtrar por el ID del "padre" que viene en la URL
      where: { restaurantId: req.params.restaurantId },
      // Ordenar (habitualmente los más nuevos primero) [cite: 329]
      order: [['createdAt', 'DESC']]
    })
    res.json(entities)
  } catch (err) {
    res.status(500).send(err.message)
  }
}
```

4. Actualización atómica (update)

```
async update (req, res) {
  try {
    // 1. Buscar la entidad por su clave primaria (PK) [cite: 352]
    const entity = await Modelo.findByPk(req.params.modeloId)
    // 2. Actualizar con los nuevos datos del body [cite: 353]
    await entity.update(req.body)
    // 3. Devolver el objeto actualizado [cite: 354]
    res.json(entity)
  } catch (err) {
    res.status(500).send(err.message)
  }
}
```

5. Borrado Seguro con Mensaje (destroy)

```
async destroy (req, res) {
  try {
    // Ejecutar el borrado basándose en el ID de la URL [cite: 360]
    const result = await Modelo.destroy({
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
      where: { id: req.params.modeloId }
    })
    // Comprobar si se ha borrado algo (result será 1 si tuvo éxito)
[cite: 362]
    const message = result === 1
      ? 'Successfully deleted.'
      : 'Could not delete.'
    res.json(message)
  } catch (err) {
    res.status(500).send(err.message)
  }
}
```

Código	Significado	Cuándo usarlo (según el PDF)
401	Unauthorized	Usuario no autenticado (isLoggedIn).
403	Forbidden	Rol incorrecto (hasRole) o no es el dueño.
404	Not Found	El restaurante o review no existe en la BD. (checkEntityExists)
409	Conflict	Lógica de negocio rota (Ya existe review, no ha pedido...).
422	Unprocessable Entity	Error de validación (Faltan campos o formato mal).
500	Internal Server Error	Error genérico de base de datos o código.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+